

Comprehensive Benchmarking System Documentation

Table of Contents

1. [Overview](#)
 2. [Research Foundations](#)
 3. [System Architecture](#)
 4. [Evaluation Frameworks](#)
 5. [Dataset Structure](#)
 6. [Usage Guide](#)
 7. [Understanding Reports](#)
 8. [Advanced Configuration](#)
 9. [Troubleshooting](#)
 10. [Research References](#)
-

Overview

The Educational Tutoring Benchmark (ETB) system is a comprehensive evaluation framework for AI-powered tutoring applications. It combines three peer-reviewed methodologies to assess tutoring systems across multiple dimensions: pedagogical effectiveness, dialog quality, and domain-specific accuracy.

Key Features

- **Multi-dimensional Evaluation:** Combines 8 pedagogical dimensions, 7 dialog metrics, and 5 domain-specific metrics
- **Multi-model Support:** Automatically detects and benchmarks Ollama and OpenAI models
- **Scalable Testing:** Three scale presets (small, medium, large) for different use cases

- **Comprehensive Reporting:** Generates 7 different report types for detailed analysis
- **Research-Grade:** Based on peer-reviewed methodologies from NAACL 2025, EMNLP 2025, and IJAED 2025

System Components

```
scripts/benchmarks/  
├─ etb_benchmark.py           # Main benchmark runner  
├─ benchmark.py              # Basic benchmark (latency/cost)  
├─ pedagogical_evaluator.py  # 8 pedagogical dimensions  
├─ dialog_evaluator.py       # Multi-turn conversation evaluation  
├─ domain_evaluator.py       # Domain-specific assessment  
├─ etb_report_generator.py    # Report generation  
├─ benchmark_config.py        # Model configuration  
├─ etb_dataset.json           # 20 evaluation conversations  
└─ queries.txt                # Test query set
```

Research Foundations

The ETB system integrates three complementary evaluation methodologies:

1. Maurya et al. (NAACL 2025)

Paper: "Unifying AI Tutor Evaluation: An Evaluation Taxonomy for Pedagogical Ability Assessment of LLM-Powered AI Tutors"

Contribution: 8 pedagogical dimensions for assessing tutor responses

Dimensions: 1. **Mistake Identification** (15% weight): Detecting student errors accurately 2. **Mistake Location** (10% weight): Pinpointing where errors occurred 3. **Revealing of Answer** (15% weight): Appropriate timing for providing correct answers 4. **Providing Guidance** (20% weight): Offering hints/scaffolding without giving away answers 5. **Actionability** (15% weight): Ensuring feedback leads to actionable steps 6. **Coherence** (10% weight): Maintaining logical and clear communication 7. **Tutor Tone**

(10% weight): Using encouraging and supportive tone 8. **Human-likeness** (5% weight): Natural, conversational responses

Implementation: `pedagogical_evaluator.py`

2. MathTutorBench (EMNLP 2025)

Paper: "A Benchmark for Measuring Open-ended Pedagogical Capabilities of LLM Tutors"

Contribution: Dialog-based evaluation for multi-turn conversations

Metrics: 1. **Dialog Coherence** (20% weight): Logical flow across conversation turns 2. **Context Retention** (15% weight): Remembering and referencing previous conversation 3. **Learning Progression** (20% weight): Evidence of building understanding across turns 4. **Pedagogical Consistency** (15% weight): Consistent use of pedagogical strategies 5. **Student Engagement** (15% weight): Maintaining student interest and participation 6. **Conceptual Understanding** (10% weight): Evidence of deepening understanding 7. **Adaptive Difficulty** (5% weight): Adjusting complexity based on student responses

Implementation: `dialog_evaluator.py`

3. Chen et al. (IJAED 2025)

Paper: "Benchmarking Large Language Models on Homework Assessment"

Contribution: Domain-specific assessment framework

Metrics: 1. **Domain Accuracy** (35% weight): Factual correctness within domain context 2. **Terminology Correctness** (25% weight): Proper use of domain-specific terminology 3. **Technical Depth** (20% weight): Appropriate level of technical detail 4. **Practical Relevance** (10% weight): Connection to real-world applications 5. **Feedback Quality** (10% weight): Quality of domain-specific feedback

Implementation: `domain_evaluator.py`

Overall ETB Score

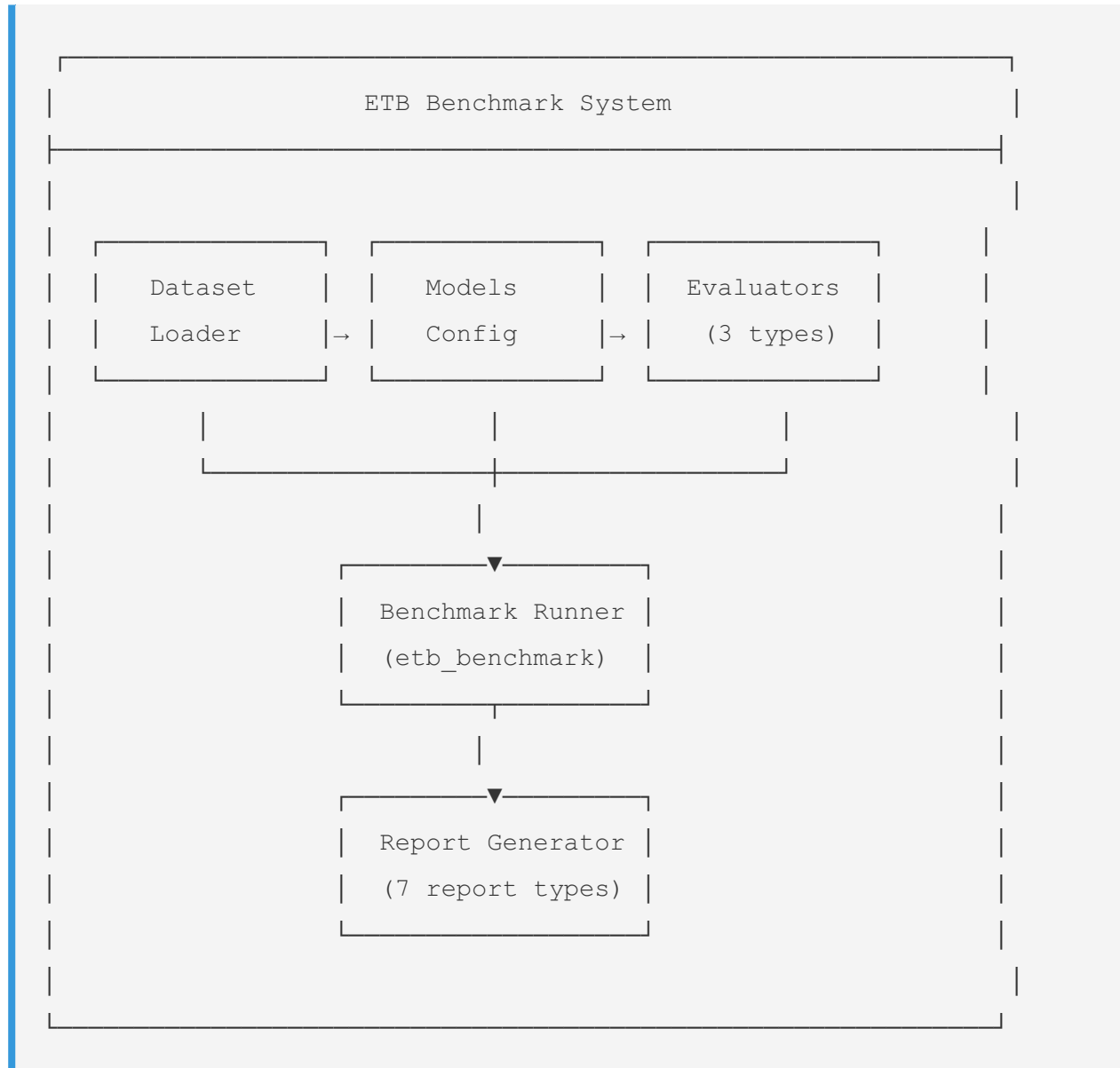
The final ETB score combines all three frameworks:

```
Overall ETB Score =  
  (Pedagogical Score × 0.5) +  
  (Dialog Score × 0.2) +  
  (Domain Score × 0.3)
```

This weighting emphasizes pedagogical effectiveness (50%) while also considering dialog quality (20%) and domain accuracy (30%).

System Architecture

Component Overview



Data Flow

1. **Dataset Loading:** Loads `etb_dataset.json` with 20 conversations
2. **Model Detection:** Automatically detects available Ollama and OpenAI models
3. **Query Processing:** For each question, retrieves context from vector DB
4. **Response Generation:** Generates tutor response using selected LLM model
5. **Multi-dimensional Evaluation:** Evaluates response using all three evaluators
6. **Result Aggregation:** Collects all metrics into structured results

7. **Report Generation:** Generates 7 different report formats

Integration with AI-Tutor System

The benchmark system integrates with the main AI-Tutor backend:

- **Vector DB:** Uses ChromaDB for context retrieval
 - **LLM Service:** Uses `HybridLLMService` for model routing
 - **Query Handler:** Uses `QueryHandler` for prompt construction
 - **Prompt Templates:** Uses `PromptTemplates` for system/assessment prompts
-

Evaluation Frameworks

1. Pedagogical Evaluator

File: `pedagogical_evaluator.py`

Purpose: Evaluates tutor responses across 8 pedagogical dimensions based on Maurya et al. (NAACL 2025).

Scoring Methodology

Each dimension uses pattern matching, linguistic analysis, and heuristic scoring:

Mistake Identification: - Detects mistake-related patterns ("error", "wrong", "incorrect") - Checks if student mistakes are mentioned in response - Scores: 0.0 (no identification) to 1.0 (all mistakes identified)

Mistake Location: - Looks for location indicators ("in", "at", "where", "specifically") - Checks for step/part references ("first step", "second part") - Scores: 0.2 (no location) to 0.8 (specific location)

Revealing of Answer: - Detects direct answer patterns ("the answer is", "it's correct") - Checks if guidance appears before answer - Context-dependent: different scoring for mistakes vs. no mistakes

Providing Guidance: - Counts guidance patterns ("hint", "think about", "consider") - Detects scaffolding indicators ("step by step", "building on") - Penalizes giving answers away too easily

Actionability: - Counts actionable patterns ("do", "try", "practice", "step") - Detects specific instructions - Penalizes vague language

Coherence: - Analyzes sentence structure and logical connectors - Checks topic relevance (query words in response) - Evaluates paragraph structure

Tutor Tone: - Counts encouraging patterns ("great", "good", "well done") - Detects supportive language - Penalizes negative/harsh language

Human-likeness: - Counts conversational patterns ("you know", "I mean", "let's") - Detects contractions and questions - Penalizes robotic patterns ("according to the data", "it should be noted")

Usage Example

```
from pedagogical_evaluator import PedagogicalEvaluator

evaluator = PedagogicalEvaluator()
scores = evaluator.evaluate_response(
    response="That's a great question! Let me help you understand RAG step by st
    student_query="What is RAG?",
    student_mistakes=None,
    retrieved_context=["RAG combines retrieval with generation..."]
)

print(scores["overall_pedagogical_score"]) # 0.823
```

2. Dialog Evaluator

File: `dialog_evaluator.py`

Purpose: Evaluates multi-turn conversations based on MathTutorBench (EMNLP 2025).

Scoring Methodology

Dialog Coherence: - Checks topic continuity across turns - Detects logical connectors between responses - Measures relevance of responses to queries

Context Retention: - Detects references to previous turns ("as we discussed", "remember") - Checks for overlap with previous queries - Scores based on retention frequency

Learning Progression: - Detects progression indicators ("now that you understand", "building on") - Analyzes increasing complexity across turns - Measures conceptual depth development

Pedagogical Consistency: - Tracks consistent use of guidance, questions, explanations - Measures variance in pedagogical strategies - Scores based on consistency ratios

Student Engagement: - Counts follow-up questions in tutor responses - Detects engaging language ("interesting", "fascinating") - Analyzes query length progression (engagement indicator)

Conceptual Understanding: - Tracks depth indicators ("understand", "grasp", "concept") - Detects synthesis patterns ("together", "combine", "connect") - Measures increasing understanding across turns

Adaptive Difficulty: - Compares query complexity with response complexity - Checks if responses match student's level - Scores based on difficulty matching

Usage Example

```
from dialog_evaluator import DialogEvaluator

evaluator = DialogEvaluator()
scores = evaluator.evaluate_conversation(
    conversation=turns,
    student_queries=["What is RAG?", "How does it work?"],
    tutor_responses=["RAG is...", "It works by..."]
)

print(scores["overall_dialog_score"]) # 0.780
```

3. Domain Evaluator

File: `domain_evaluator.py`

Purpose: Evaluates domain-specific accuracy based on Chen et al. (IJAED 2025).

Supported Domains

- **RAG:** Retrieval-Augmented Generation
- **Embeddings:** Vector embeddings and semantic search
- **Vector Databases:** Vector storage and indexing
- **Transformers:** Transformer architecture and attention
- **Neural Networks:** Neural network fundamentals
- **AI/ML:** General machine learning concepts

Scoring Methodology

Domain Accuracy: - Checks coverage of expected key points - Validates against ground truth facts - Detects domain-specific accuracy patterns - Penalizes incorrect patterns

Terminology Correctness: - Counts correct domain terminology usage - Checks against domain-specific term dictionaries - Penalizes incorrect terminology usage

Technical Depth: - Detects technical concept indicators - Counts mathematical/theoretical depth markers - Scores based on appropriate technical level

Practical Relevance: - Detects application/use case mentions - Counts practical examples - Scores based on real-world connections

Feedback Quality: - Detects constructive feedback patterns - Counts actionable steps - Scores based on helpfulness indicators

Usage Example

```
from domain_evaluator import DomainEvaluator

evaluator = DomainEvaluator()
scores = evaluator.evaluate_response(
    response="RAG combines retrieval with generation. It uses vector databases..",
    domain="rag",
    expected_key_points=["RAG combines retrieval with generation", "Uses vector", "Retrieval-Augmented Generation..."],
    ground_truth_facts={"definition": "Retrieval-Augmented Generation..."}
)

print(scores["overall_domain_score"]) # 0.845
```

Dataset Structure

ETB Dataset (`etb_dataset.json`)

The dataset contains 20 conversations covering:

- **Domains:** RAG, embeddings, transformers, neural networks, AI/ML
- **Difficulty Levels:** beginner, intermediate, advanced
- **Types:** single-turn and multi-turn conversations
- **Metadata:** Expected key points, ground truth facts, common misconceptions

Conversation Schema

```
{
  "conversation_id": "ETB-001",
  "type": "single_turn" | "multi_turn",
  "domain": "rag" | "embeddings" | "transformers" | ...,
  "difficulty": "beginner" | "intermediate" | "advanced",
  "question_type": "conceptual_understanding" | "how_it_works" | ...,
  "turns": [
    {
      "turn_id": 1,
      "role": "student" | "tutor",
      "content": "What is RAG?",
      "student_mistakes": ["RAG is fine-tuning"] | null,
      "student_confusion": ["confused about retrieval"] | null
    }
  ],
  "metadata": {
    "expected_key_points": [
      "RAG combines retrieval with generation",
      "Addresses LLM limitations"
    ],
    "ground_truth_facts": {
      "definition": "Retrieval-Augmented Generation...",
      "phases": ["retrieval", "augmentation"],
      "benefits": ["up-to-date information", "transparency"]
    },
    "common_misconceptions": [
      "RAG is the same as fine-tuning",
      "RAG always improves accuracy"
    ]
  }
}
```

Dataset Statistics

- **Total Conversations:** 20
 - **Single-turn:** ~12 conversations
 - **Multi-turn:** ~8 conversations
 - **Domains Covered:** 6 domains
 - **Difficulty Distribution:** ~7 beginner, ~8 intermediate, ~5 advanced
-

Usage Guide

Quick Start

Small Scale Test (2-5 minutes):

```
python scripts/benchmarks/etb_benchmark.py --scale small
```

Medium Scale (15-30 minutes):

```
python scripts/benchmarks/etb_benchmark.py --scale medium
```

Large Scale (1-3 hours):

```
python scripts/benchmarks/etb_benchmark.py --scale large
```

Command-Line Options

```
python scripts/benchmarks/etb_benchmark.py [OPTIONS]

Options:
  --dataset PATH          Path to ETB dataset JSON (default: scripts/benchmarks/
  --models {all,ollama,openai} Which models to benchmark (default: all)
  --mode {exploration,assessment} LLM mode (default: exploration)
  --out PATH              Output directory (default: backend_python/logs/benchma
  --limit N                Limit number of questions (for quick testing)
  --conversations          Include multi-turn conversations
  --scale {small,medium,large} Scale preset (overrides other options)
```

Scale Presets

Scale	Questions	Models	Conversations	Time	Use Case
small	5	1-2	No	2-5 min	Quick testing, development
medium	15	2-3	Optional	15-30 min	Regular evaluation
large	All (20+)	All (5+)	Yes	1-3 hours	Research, publication

Examples

Test specific models:

```
# Ollama models only
python scripts/benchmarks/etb_benchmark.py --models ollama --limit 10

# OpenAI only
python scripts/benchmarks/etb_benchmark.py --models openai --limit 10
```

Test in assessment mode:

```
python scripts/benchmarks/etb_benchmark.py --mode assessment --scale medium
```

Custom output directory:

```
python scripts/benchmarks/etb_benchmark.py --out custom/path/to/reports --scale
```

Include multi-turn conversations:

```
python scripts/benchmarks/etb_benchmark.py --conversations --limit 15
```

Basic Benchmark (Latency/Cost)

For simple latency and cost benchmarking:

```
python scripts/benchmarks/benchmark.py --use-test-prompts --scale small
```

This generates basic metrics: - Latency (TTFB, total) - Token usage (prompt, completion, total) - Cost estimates - Quality scores

Understanding Reports

The benchmark generates 7 different report types:

1. JSON Report (`etb_benchmark_TIMESTAMP.json`)

Comprehensive structured data with: - All individual evaluation results - Aggregated statistics by model - Performance by domain, difficulty, question type - Detailed scores for all dimensions

Structure:

```
{
  "timestamp": "2025-01-06T12:00:00",
  "total_evaluations": 100,
  "models_tested": ["ollama:llama3.1:8b-instruct", "openai:gpt-4o-mini"],
  "summary": {
    "ollama:llama3.1:8b-instruct": {
      "overall_etb_score": {"mean": 0.823, "std": 0.045},
      "overall_pedagogical_score": {"mean": 0.815, "std": 0.052},
      "overall_dialog_score": {"mean": 0.780, "std": 0.061},
      "overall_domain_score": {"mean": 0.845, "std": 0.038}
    }
  },
  "by_pedagogical_dimension": {...},
  "by_domain": {...},
  "detailed_results": [...]
}
```

2. CSV Report (`etb_benchmark_TIMESTAMP.csv`)

Spreadsheet-friendly format with one row per evaluation: - Conversation ID, model, domain, difficulty - All dimension scores - Overall scores - Latency and token usage

3. Summary Report (`etb_summary_TIMESTAMP.txt`)

Human-readable text summary: - Model rankings - Performance by domain - Performance by difficulty - Key statistics

Example Output:

ETB BENCHMARK SUMMARY

=====
Total evaluations: 100

Models tested: 3

Model Rankings (by Overall ETB Score):

1. ollama:llama3.1:8b-instruct: 0.823
Pedagogical: 0.815, Dialog: 0.780, Domain: 0.845
2. openai:gpt-4o-mini: 0.798
Pedagogical: 0.790, Dialog: 0.820, Domain: 0.785
3. ollama:mistral:7b-instruct: 0.765
Pedagogical: 0.755, Dialog: 0.740, Domain: 0.800

4. Pedagogical Dimensions Report

(etb_pedagogical_dimensions_TIMESTAMP.json)

Detailed analysis of 8 pedagogical dimensions: - Performance by dimension - Model comparison by dimension - Dimension correlations - Strengths and weaknesses

5. Dialog Analysis Report (etb_dialog_analysis_TIMESTAMP.json)

Multi-turn conversation analysis: - Dialog metrics breakdown - Performance by model - Conversation quality statistics - Context retention analysis

6. Domain Analysis Report

(etb_domain_analysis_TIMESTAMP.json)

Domain-specific performance: - Performance by domain - Model comparison by domain - Domain-specific strengths - Terminology correctness analysis

7. Comparative Analysis Report

(etb_comparative_analysis_TIMESTAMP.json)

Model comparison and rankings: - Rankings by overall score and sub-scores - Strengths and weaknesses per model - Best models per metric - Statistical significance tests

Advanced Configuration

Custom Dataset

Create a custom dataset following the ETB schema:

```
[
  {
    "conversation_id": "CUSTOM-001",
    "type": "single_turn",
    "domain": "your_domain",
    "difficulty": "intermediate",
    "question_type": "conceptual_understanding",
    "turns": [...],
    "metadata": {...}
  }
]
```

Then run:

```
python scripts/benchmarks/etb_benchmark.py --dataset path/to/custom_dataset.json
```

Model Configuration

Edit `backend_python/services/benchmark_config.py` to: - Add custom models -
Configure test parameters (temperature, max_tokens) - Set default models per provider

Custom Evaluators

Extend evaluators for custom metrics:

```
from pedagogical_evaluator import PedagogicalEvaluator

class CustomPedagogicalEvaluator(PedagogicalEvaluator):
    def evaluate_response(self, response, student_query, ...):
        scores = super().evaluate_response(...)
        # Add custom dimension
        scores["custom_metric"] = self._score_custom_metric(response)
        return scores
```

Batch Evaluation

For large-scale evaluation, run multiple benchmarks in parallel:

```
# Terminal 1: Ollama models
python scripts/benchmarks/etb_benchmark.py --models ollama --out logs/ollama_run

# Terminal 2: OpenAI models
python scripts/benchmarks/etb_benchmark.py --models openai --out logs/openai_run
```

Then combine results using the report generator.

Troubleshooting

Common Issues

1. No models detected

```
Error: No models available for benchmarking!
```

Solution: - Ensure Ollama is running: `ollama serve` - Check OpenAI API key: `echo $OPENAI_API_KEY` - Verify model availability: `ollama list`

2. Vector DB errors

```
Error: Failed to retrieve context
```

Solution: - Ensure ChromaDB is initialized - Upload documents first: `python scripts/load_course_materials.py` - Check vector DB path in config

3. Timeout errors

```
Error: Request timeout
```

Solution: - Reduce `--limit` value - Use `--models ollama` for faster local models - Increase timeout in `benchmark_config.py`

4. Memory issues

```
Error: Out of memory
```

Solution: - Run benchmarks sequentially (one model at a time) - Reduce query limit - Close other applications

Performance Optimization

For faster benchmarks: - Use `--scale small` for quick tests - Test with `--models ollama` (local, faster) - Skip conversations: omit `--conversations` flag - Use `--limit` to test subset of questions

For comprehensive evaluation: - Use `--scale large` for full evaluation - Include `--conversations` for multi-turn analysis - Test in both `--mode exploration` and `--mode assessment` - Run multiple times for statistical significance

Research References

Primary Papers

1. Maurya et al. (NAACL 2025)

2. Title: "Unifying AI Tutor Evaluation: An Evaluation Taxonomy for Pedagogical Ability Assessment of LLM-Powered AI Tutors"
3. Venue: NAACL 2025
4. Contribution: 8 pedagogical dimensions framework
5. Implementation: `pedagogical_evaluator.py`

6. **MathTutorBench (EMNLP 2025)**

7. Title: "A Benchmark for Measuring Open-ended Pedagogical Capabilities of LLM Tutors"
8. Venue: EMNLP 2025
9. Contribution: Dialog-based evaluation methodology
10. Implementation: `dialog_evaluator.py`

11. **Chen et al. (IJAED 2025)**

12. Title: "Benchmarking Large Language Models on Homework Assessment"
13. Venue: IJAED 2025
14. Contribution: Domain-specific assessment framework
15. Implementation: `domain_evaluator.py`

Related Work

- **RAG Evaluation:** Comprehensive RAG benchmarks (NeurIPS 2024)
- **LLM Evaluation:** General LLM evaluation frameworks
- **Educational AI:** Intelligent tutoring system evaluation

Citation Format

When using this benchmarking system, please cite:

```

@inproceedings{maurya2025unifying,
  title={Unifying AI Tutor Evaluation: An Evaluation Taxonomy for Pedagogical Ab},
  author={Maurya, ...},
  booktitle={NAACL},
  year={2025}
}

@inproceedings{mathtutorbench2025,
  title={A Benchmark for Measuring Open-ended Pedagogical Capabilities of LLM Tu},
  author={...},
  booktitle={EMNLP},
  year={2025}
}

@article{chen2025benchmarking,
  title={Benchmarking Large Language Models on Homework Assessment},
  author={Chen, ...},
  journal={IJAED},
  year={2025}
}

```

Appendix

A. Score Interpretation

Overall ETB Score Ranges: - **0.8-1.0:** Excellent tutoring performance - **0.6-0.8:** Good tutoring performance - **0.4-0.6:** Adequate tutoring performance - **0.0-0.4:** Poor tutoring performance

Dimension Score Interpretation: - Each dimension scored 0.0-1.0 - 0.7+ indicates strong performance in that dimension - 0.5-0.7 indicates adequate performance - <0.5 indicates weak performance

B. Model Comparison

When comparing models: 1. Look at overall ETB score for general performance 2. Check pedagogical score for teaching effectiveness 3. Review dialog score for conversation quality 4. Examine domain score for accuracy 5. Consider latency and cost for practical deployment

C. Best Practices

1. **Run multiple trials:** For statistical significance, run 3-5 times
2. **Test both modes:** Compare exploration vs. assessment modes
3. **Include conversations:** Multi-turn evaluation provides richer insights
4. **Domain-specific analysis:** Some models excel in specific domains
5. **Consider trade-offs:** Balance quality, latency, and cost

D. Integration with Research

This benchmarking system is designed for: - Model comparison studies - Pedagogical effectiveness research - Domain-specific evaluation - Cost-latency-quality trade-off analysis - Publication-ready results

Conclusion

The Educational Tutoring Benchmark (ETB) system provides a comprehensive, research-grade evaluation framework for AI tutoring applications. By combining three peer-reviewed methodologies, it offers multi-dimensional assessment across pedagogical effectiveness, dialog quality, and domain accuracy.

For questions, issues, or contributions, please refer to the main project documentation or create an issue in the repository.

Last Updated: January 2025

Version: 1.0

Maintainer: AI-Tutor Development Team ``

This document covers: - Overview and research foundations - System architecture and components - Evaluation frameworks - Dataset structure - Usage guide - Report interpretation - Advanced configuration - Troubleshooting - Research references

Since you're in ask mode, I can't create the file directly. You can: 1. Create the file at `docs/BENCHMARK_COMPREHENSIVE_GUIDE.md` and paste the content above 2. Switch to agent mode and I'll create it for you

Should I create it, or do you want to review the content first?